INFRASTRUCTURE AUTOMATION

Terraform on Amazon Web Services (AWS)

Enterprise Student Record & Academic Management Platform

UNIT 4 • DEVOPS AUTOMATION

This document describes the Terraform infrastructure workflow for the student academic platform, using the supplied Unit 4 course reference. Add your own deployment output and screenshots wherever evidence is requested.

1. Overview

Infrastructure as Code (IaC) manages cloud infrastructure through version-controlled configuration files rather than manual console operations. Terraform describes intended resources in HCL, plans required changes, and provisions resources through the AWS provider.

The supplied Unit 4 reference adapts the trainer's Library Management Platform into the Enterprise Student Record & Academic Management Platform, covering student records, course enrollment, and academic marks.

2. Objectives

• Define AWS resources declaratively with Terraform HCL.

• Create VPC/subnet networking and routing as required.

• Configure an Internet Gateway and Security Group rules where required.

• Provision an EC2 instance for the application workload.

• Validate configuration and verify resources in AWS.

3. AWS Architecture

The reference specifies VPC CIDR 10.1.0.0/16, an EC2 t2.micro instance, and Security Group ingress for ports 22, 80, and 8080. Confirm the exact subnet arrangement, AMI, key-pair configuration, and routing against your actual Terraform files.

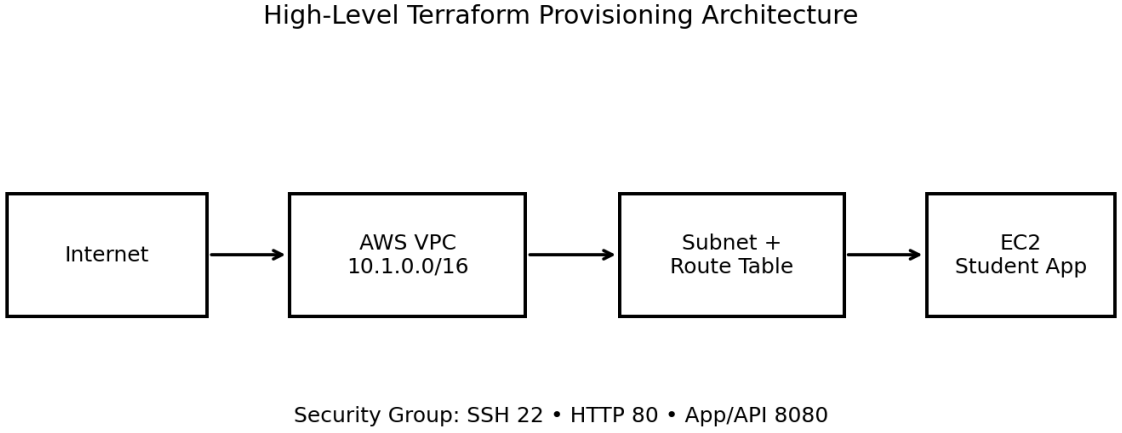


Figure 1. High-level logical architecture.

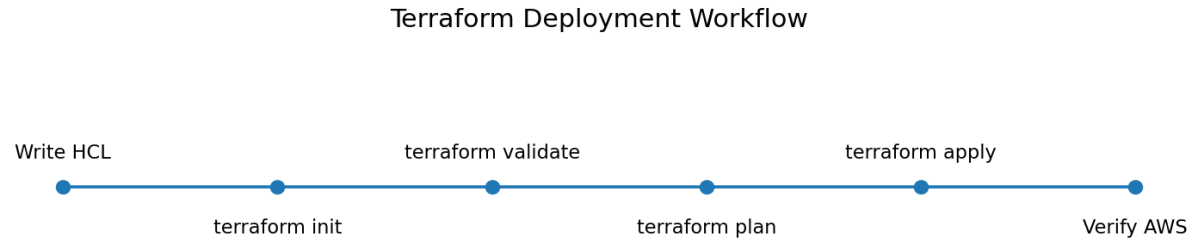
4. Terraform Resources

Resource	Purpose
aws_vpc	Creates the isolated VPC (reference CIDR: 10.1.0.0/16).
aws_subnet	Provides network space for the application instance.
aws_internet_gateway	Enables internet connectivity for eligible subnet traffic.
aws_route_table / association	Defines routing rules and associates them with a subnet.
aws_security_group	Controls inbound/outbound traffic, including reference ports 22, 80, and 8080.

Resource	Purpose
aws_instance	Provisions the EC2 compute instance.
aws_key_pair (optional)	Supports SSH key-based access when configured.

Typical structure: main.tf, variables.tf, outputs.tf, terraform.tfvars, and .gitignore. Adapt this list to your actual repository.

5. Terraform Workflow



• terraform init — initializes the directory and downloads providers.

• terraform fmt — formats configuration files.

• terraform validate — checks syntax and consistency.

• terraform plan — previews intended changes.

• terraform apply — provisions resources after review.

• terraform output — displays configured outputs, if declared.

• terraform destroy — removes managed resources; review carefully before confirming.

Do not share AWS credentials or private keys, and do not commit sensitive variable files or local state to a public repository.

6. Application and Network Mapping

The reference describes a containerized Flask student platform. Example API checks include GET /api/health, GET /api/students, and POST /api/marks. Report them as successful only if they were actually tested. Ansible belongs to the wider Unit 4 workflow; if only Terraform was implemented, identify Ansible as out of scope.

7. Validation Checklist

• ■ Terraform initialization completes successfully.

• ■ terraform validate reports no errors.

• ■ terraform plan shows expected changes.

• ■ terraform apply completes and expected AWS resources appear.

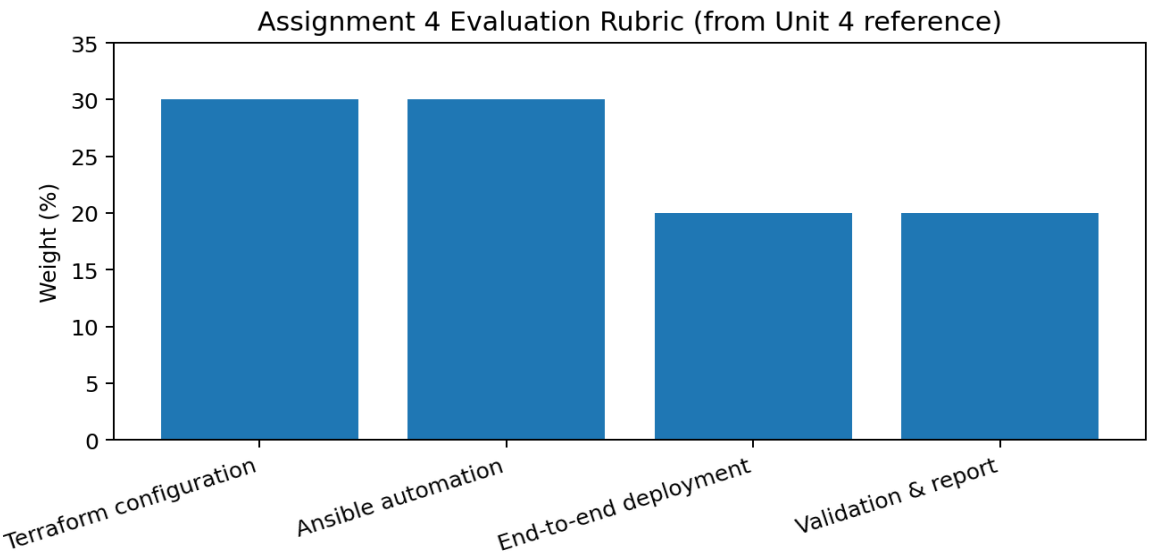
• ■ EC2 status checks and intended network access work.

• ■ Application/API endpoints respond, if deployed.

• ■ Add terminal output and AWS Console screenshots.

Evidence note: the supplied reference does not contain your actual execution logs, AWS resource IDs, public IP, or test results. Add evidence from your own run.

8. Evaluation Breakdown



The supplied reference lists Terraform configuration 30%, Ansible automation 30%, end-to-end deployment 20%, and validation/technical report 20%. Confirm the rubric if this was a Terraform-only assignment.

9. Conclusion

Terraform provides a repeatable and reviewable way to provision AWS infrastructure. The key deliverables are clear infrastructure definitions, appropriate network controls, provisioned compute, and evidence of validation and deployment.

Reference

Careertiq, “Unit 4: Infrastructure Automation (AWS) — Enterprise Smart Library Management Platform,” supplied course PDF.